V&G

Manuel d'utilisation

Présentation de l'application et parcours par rôle

Projet : application web du traiteur Vite & Gourmand (Bordeaux)

Candidat : Mahmoud EL BRAIKI

Organisme : Studi — Évaluation en cours de formation

Date : Octobre 2026

Manuel d’utilisation

1. Présentation de l’application

Vite & Gourmand est l’application web du traiteur bordelais de Julie et José. Elle permet :

- aux **visiteurs** de découvrir l’entreprise, les avis clients et l’ensemble des menus, de les filtrer et de contacter l’entreprise ;
- aux **utilisateurs** (clients inscrits) de commander un menu, de suivre, modifier ou annuler leurs commandes et de déposer un avis ;
- aux **employés** de traiter les commandes, de gérer les menus, les plats, les horaires et de modérer les avis ;
- à l’**administrateur** (José) de faire tout ce que fait un employé, de gérer les comptes employés et de consulter les statistiques de commandes et le chiffre d’affaires.

L’application est responsive : elle s’utilise sur ordinateur, tablette et smartphone.

2. Identifiants de test

Rôle	Identifiant (e-mail)	Mot de passe
Administrateur	jose@vitegourmand.fr	Admin!Gourmand2026
Employée	julie@vitegourmand.fr	Employe!Gourmand2026
Utilisateur	client@test.fr	Client!Gourmand2026
Employé désactivé (démonstration)	ancien.employe@vitegourmand.fr	Employe!Gourmand2026

Vous pouvez aussi créer votre propre compte utilisateur depuis le bouton **Connexion → Créer un compte**.

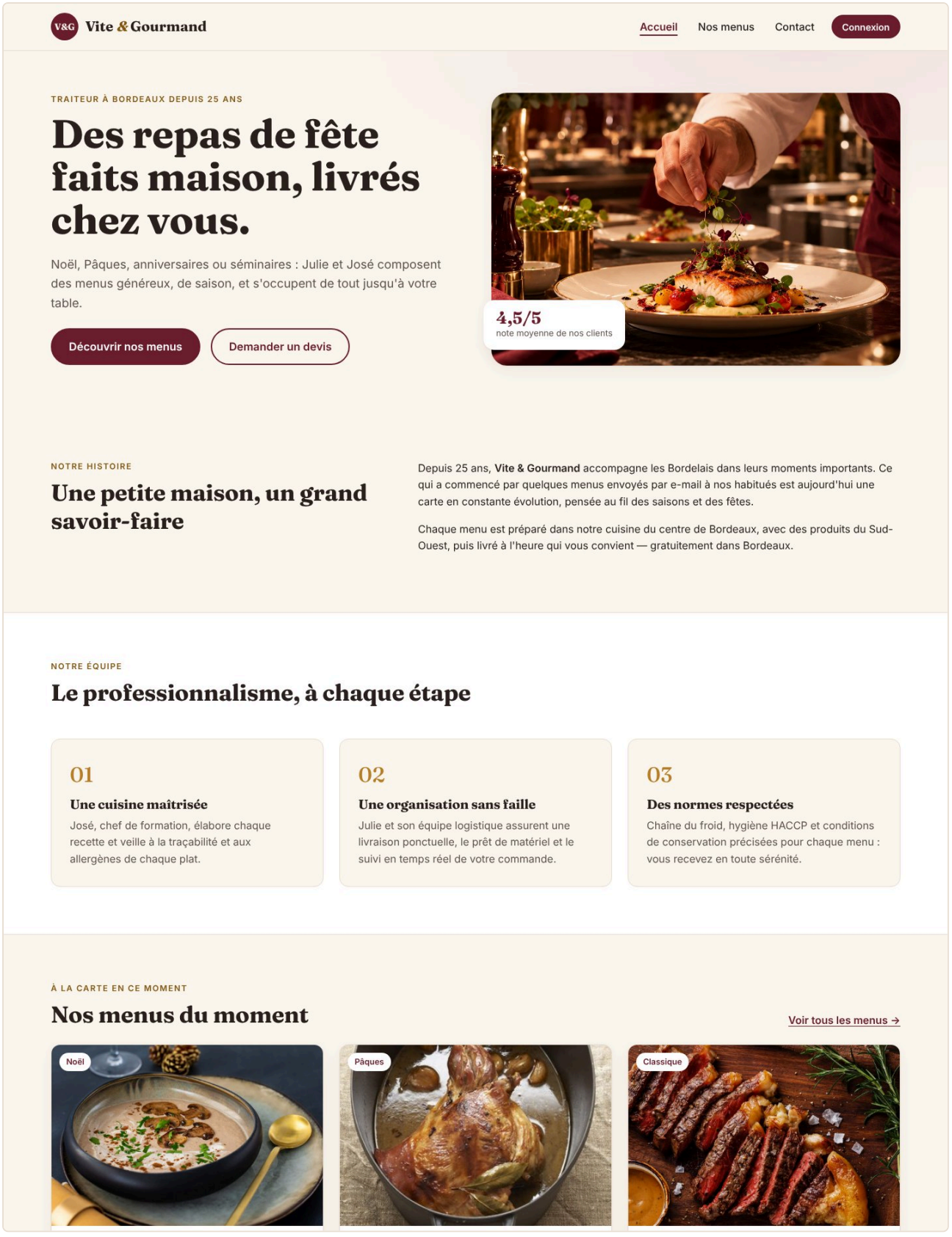
3. Navigation générale

Le menu principal, présent sur toutes les pages, donne accès à :

- **Accueil** — présentation de l’entreprise, professionnalisme de l’équipe, menus du moment et avis validés ;
- **Nos menus** — vue globale de tous les menus ;
- **Contact** — formulaire de contact ;
- **Connexion** (ou **Mon espace / Espace employé / Administration** et **Déconnexion** une fois connecté).

Le pied de page affiche les **horaires du lundi au dimanche** ainsi que les liens vers les **mentions légales**, les **conditions générales de vente** et la **politique de confidentialité**.

Sur mobile, le menu est replié derrière le bouton « hamburger » en haut à droite.

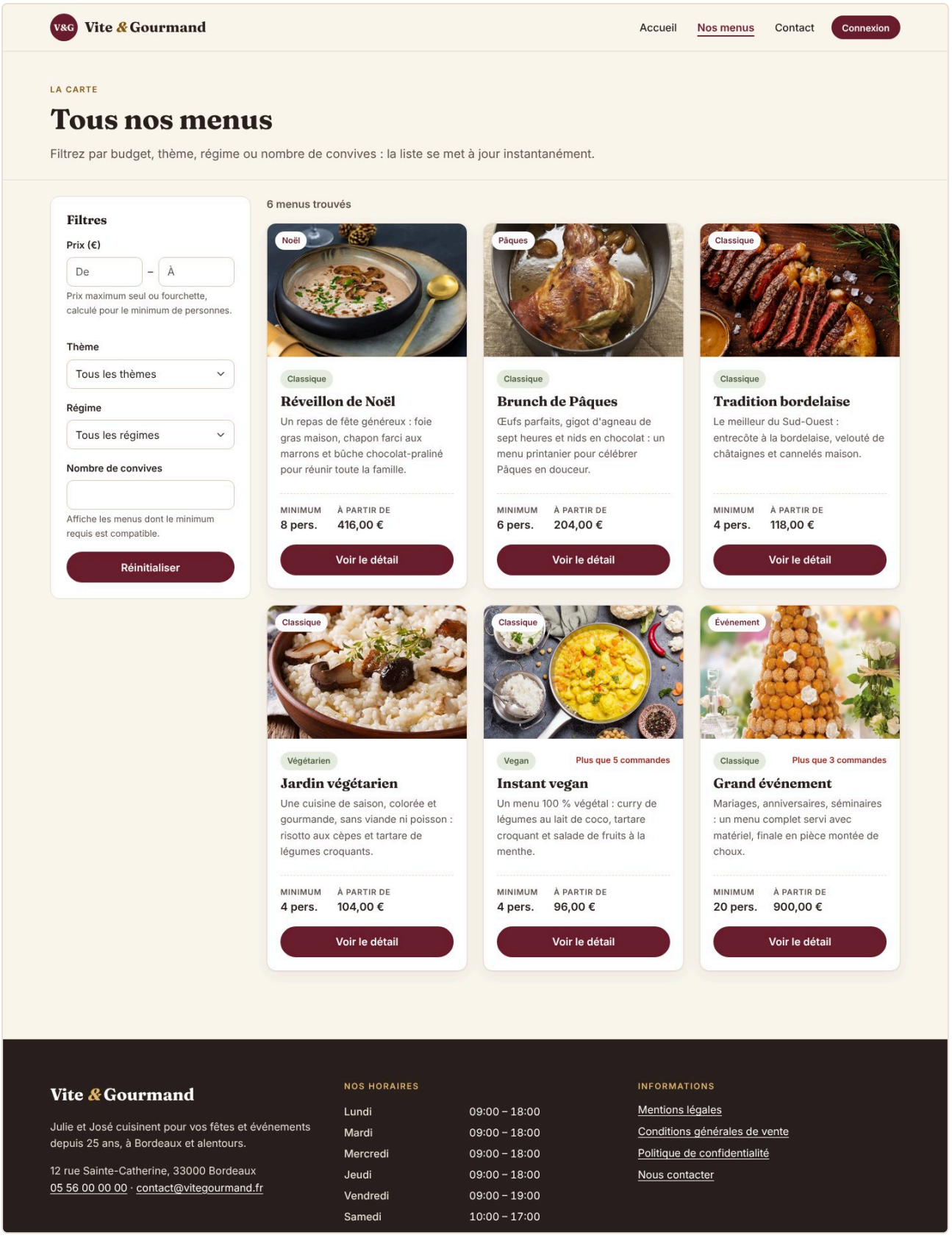


Accueil sur ordinateur

4. Parcours visiteur

4.1 Consulter et filtrer les menus

1. Cliquez sur **Nos menus**.
2. Chaque carte présente le titre, la description, le nombre minimum de personnes et le prix pour ce minimum.
3. Utilisez les filtres à gauche : **prix maximum** ou **fourchette de prix**, **thème**, **régime**, **nombre de convives**. La liste se met à jour immédiatement, **sans rechargement de la page**, et le nombre de résultats est annoncé.
4. **Réinitialiser** efface tous les filtres.



Vue globale des menus et filtres

4.2 Voir le détail d'un menu

Le bouton **Voir le détail** affiche la galerie d'images (cliquez sur une vignette pour l'agrandir), la composition (entrées, plats, desserts) avec les **allergènes** de chaque plat, le prix, le stock restant et, dans un encadré orange très visible, les **conditions du menu** (délai de commande, conservation, prêt de matériel).

4.3 Contacter l'entreprise

Page **Contact** : saisissez un titre, une description et votre e-mail. La demande est envoyée par e-mail à l'entreprise, qui vous répond à l'adresse indiquée.

5. Parcours utilisateur

5.1 Créer un compte

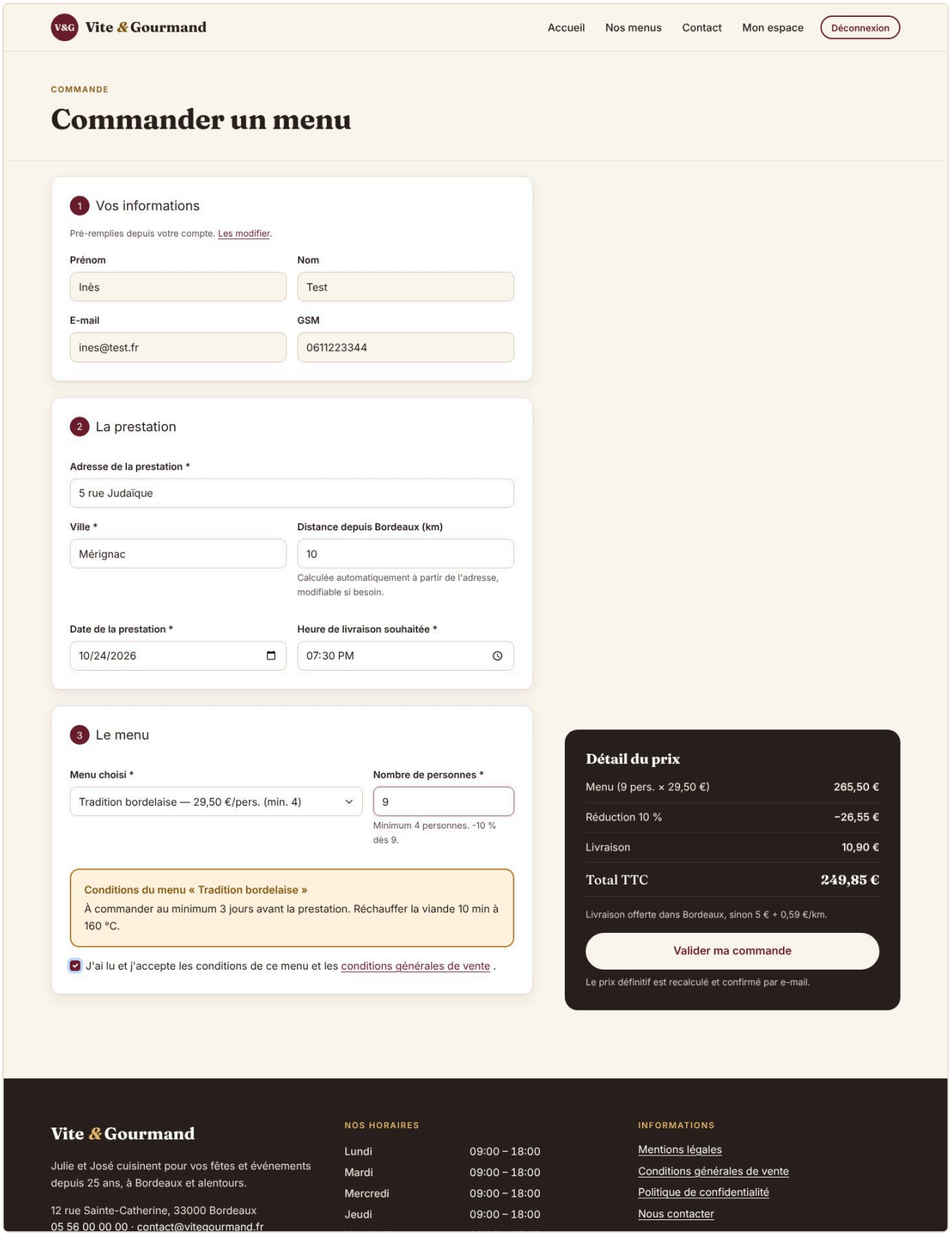
Connexion → Créer un compte : nom, prénom, GSM, e-mail, adresse postale, ville et un mot de passe sécurisé (**10 caractères minimum, avec une majuscule, une minuscule, un chiffre et un caractère spécial** — les règles se cochent en direct). Après validation, vous êtes connecté(e) avec le rôle « utilisateur » et recevez un **e-mail de bienvenue**.

5.2 Se connecter / mot de passe oublié

Saisissez votre e-mail et votre mot de passe. En cas d'oubli, **Mot de passe oublié ?** envoie un lien de réinitialisation valable 1 heure.

5.3 Commander un menu

1. Depuis le détail d'un menu, cliquez sur **Commander ce menu** (si vous n'êtes pas connecté(e), l'application vous demande de vous connecter ou de créer un compte, puis vous ramène à la commande).
2. Vos nom, prénom, e-mail et GSM sont **pré-remplis** depuis votre compte.
3. Renseignez l'adresse, la ville, la date et l'heure de livraison. Hors Bordeaux, la **distance** est estimée automatiquement à partir de l'adresse (modifiable).
4. Le menu est déjà sélectionné ; choisissez le **nombre de personnes** (au moins le minimum du menu).
5. Le panneau **Détail du prix** se met à jour : prix du menu, **réduction de 10 %** dès 5 personnes de plus que le minimum, **livraison** (offerte à Bordeaux, sinon 5 € + 0,59 €/km) et total.
6. Cochez la case confirmant la lecture des conditions du menu puis **Valider ma commande**. Un **e-mail de confirmation** vous est envoyé.



Page de commande

5.4 Mon espace

- **Mes commandes** : liste de toutes vos commandes et leur statut ; **Détail** affiche toutes les informations.
- Tant que la commande est **« en attente »**, vous pouvez la **modifier** (tout sauf le menu) ou l'**annuler**.
- Dès qu'elle est **« acceptée »**, le **suivi** liste chaque étape avec la date et l'heure.
- Quand la commande est **« terminée »**, vous êtes invité(e) par e-mail à donner un **avis** : note de 1 à 5 et commentaire, publié sur l'accueil après validation.
- **Mes informations** : modification des coordonnées et du mot de passe.



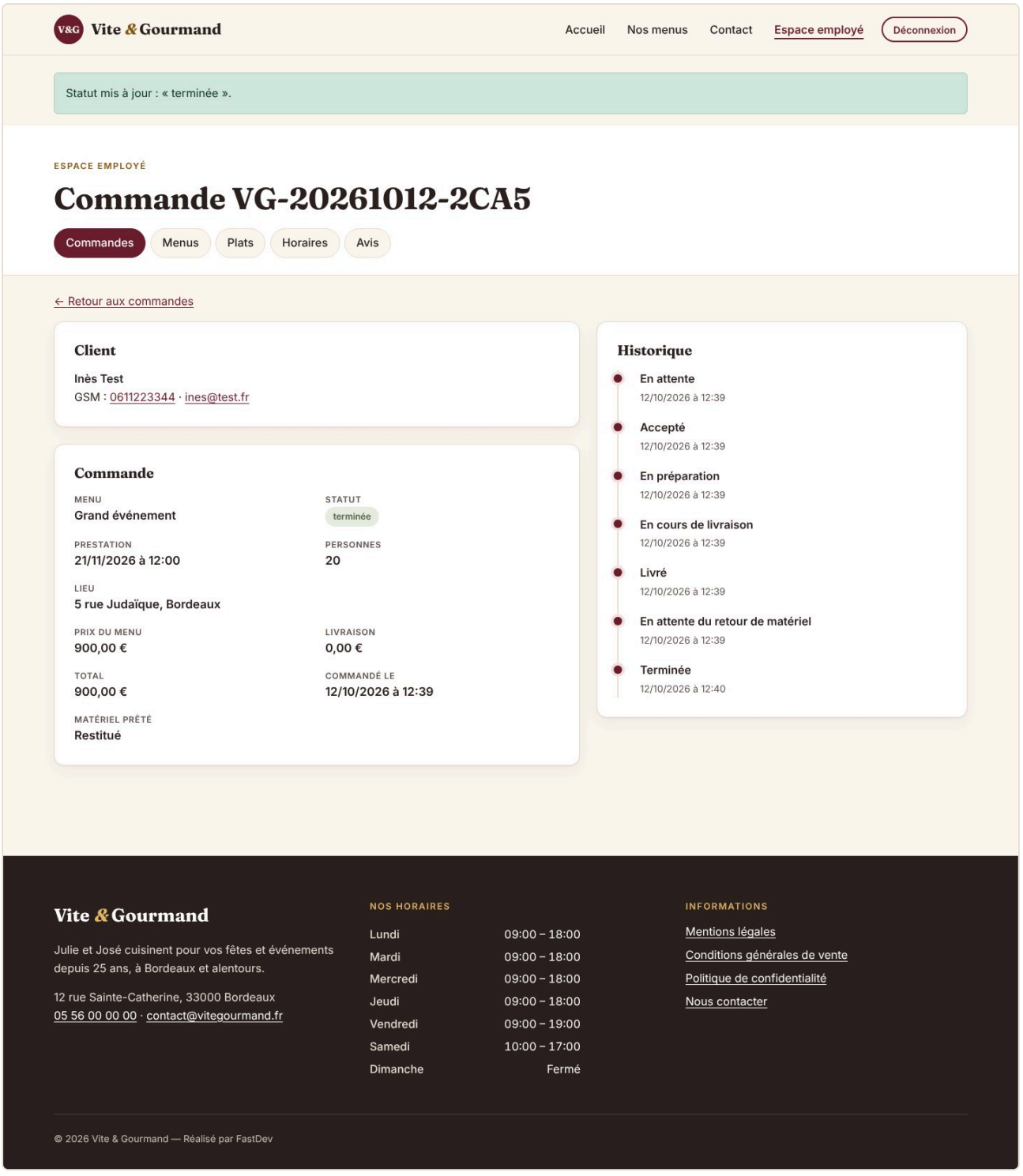
Suivi d'une commande sur mobile

6. Parcours employé

Connectez-vous avec un compte employé : vous arrivez sur **Espace employé → Commandes**.

6.1 Gérer les commandes

- Les compteurs en haut donnent le nombre de commandes par statut (cliquables).
- Le **filtre** permet de retrouver les commandes par **statut** et/ou par **client** (nom, prénom ou e-mail).
- **Gérer** ouvre la commande : coordonnées du client (GSM et e-mail cliquables), détail et historique.
- **Mettre à jour le statut** : accepté → en préparation → en cours de livraison → livré → *en attente du retour de matériel* → terminée. Cochez « du matériel a été prêté » si nécessaire : le statut « en attente du retour de matériel » envoie automatiquement au client l'e-mail l'informant des **600 € de frais** sans restitution sous 10 jours ouvrés ; la commande ne peut être terminée qu'une fois le matériel **restitué**.
- **Modifier** ou **Annuler** une commande n'est possible qu'**après avoir contacté le client** : le mode de contact (GSM ou e-mail) et le motif sont obligatoires.



Gestion d'une commande

6.2 Menus, plats, horaires, avis

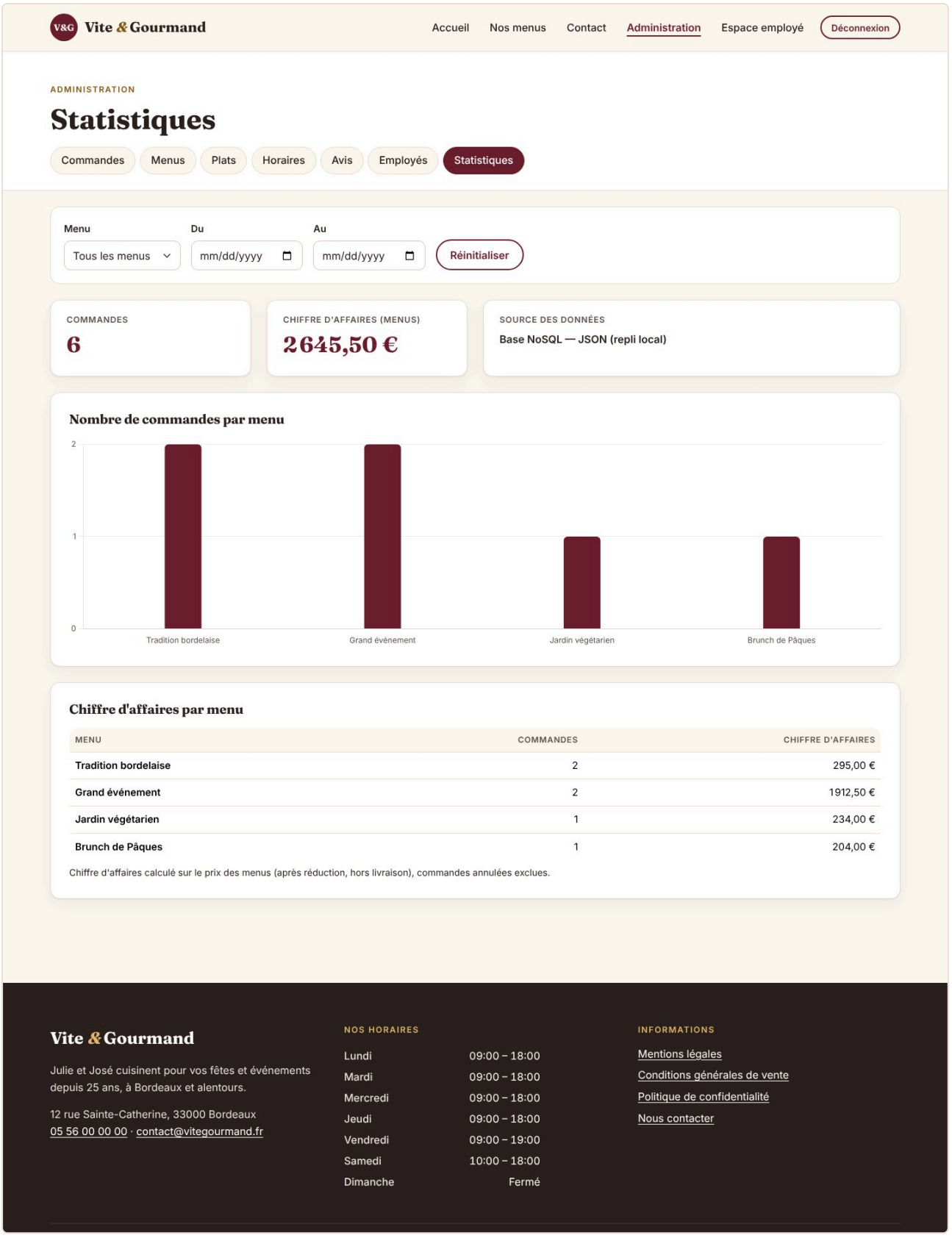
- **Menus** : créer, modifier (titre, description, conditions, thème, régime, minimum, prix, stock, visibilité, plats proposés, galerie d'images) ou supprimer un menu. Un menu déjà commandé est **désactivé** plutôt que supprimé pour conserver l'historique. Des régimes et thèmes peuvent être ajoutés.
- **Plats** : créer, modifier ou supprimer un plat (type, photo, allergènes). Un même plat peut figurer dans plusieurs menus.

- **Horaires** : modifier les horaires du lundi au dimanche (affichés dans le pied de page).
- **Avis** : **valider** un avis pour le publier sur l'accueil, ou le **refuser**.

7. Parcours administrateur

Le compte administrateur dispose de tout l'espace employé, plus :

- **Employés** : créer un compte employé (e-mail = identifiant + mot de passe). L'employé reçoit un e-mail l'informant de la création de son compte, **sans le mot de passe** : l'administrateur le lui communique en main propre. **Désactiver** rend le compte inutilisable (départ de l'entreprise) ; il peut être réactivé. Il est impossible de créer un administrateur depuis l'application.
- **Statistiques** : graphique comparant le **nombre de commandes par menu** et tableau du **chiffre d'affaires par menu**, filtrables par **menu** et par **période** (du… au…). Ces données proviennent de la **base NoSQL**.



Statistiques administrateur

8. Récapitulatif des parcours de test

Parcours	Compte	Étapes
Découverte	aucun	Accueil → Nos menus → filtres → Voir le détail
Inscription et commande	nouveau compte	Créer un compte → Commander un menu → Mon espace → Modifier / Annuler
Suivi et avis	client@test.fr	Mon espace → commande VG-20260815-A1B2 (terminée, avis déjà déposé)
Traitement d'une commande	julie@vitegourmand.fr	Commandes → filtre « en attente » → Gérer → statuts successifs
Retour de matériel	julie@vitegourmand.fr	Commande VG-20260918-E5F6 → cocher « matériel restitué » → Terminée
Modération	julie@vitegourmand.fr	Avis → Valider / Refuser
Administration	jose@vitegourmand.fr	Employés → créer / désactiver → Statistiques → filtres

V&G

Charte graphique

Palette, typographie, wireframes et maquettes

Projet : application web du traiteur Vite & Gourmand (Bordeaux)

Candidat : Mahmoud EL BRAIKI

Organisme : Studi — Évaluation en cours de formation

Date : Octobre 2026

Charte graphique

1. Intention

Vite & Gourmand est une petite maison bordelaise, artisanale et chaleureuse, qui travaille depuis 25 ans pour les grandes occasions. L'identité visuelle traduit trois idées :

- **la table de fête** : un bordeaux profond (le vin, la région), un fond crème (la nappe) et un ocre doré (le cannelé, la croûte d'un pain) ;
- **la confiance** : une interface claire, aérée et lisible, où les informations importantes (prix, conditions, allergènes) sont toujours visibles ;
- **l'accessibilité** : tous les couples texte / fond respectent le niveau AA du RGAA (contraste ≥ 4,5:1).

2. Palette de couleurs



Palette de couleurs et contrastes

Couleur	Code	Usage	Contraste
Bordeaux	#6B1E2E	Boutons, liens, accents	11,4:1 avec du texte blanc
Bordeaux foncé	#521522	Survol des boutons	14,1:1
Crème	#FBF6EE	Fond général	—

Couleur	Code	Usage	Contraste
Blanc	#FFFFFF	Cartes, formulaires	—
Encre	#2B2220	Texte principal	14,4:1 sur crème
Encre secondaire	#5F5450	Textes secondaires	6,8:1 sur crème
Ocre	#B9802A	Décor, étoiles, contour de focus	3,2:1 (éléments non textuels, seuil 3:1)
Ocre texte	#855A14	Surtitres, encadré « conditions »	5,6:1 sur crème
Sauge	#4F6142	Régimes, statut « terminée »	6,3:1 sur crème
Rouge alerte	#A1281F	Annulation, stock faible	6,9:1 sur crème

L'ocre décoratif n'est jamais utilisé pour du texte : une variante plus sombre (« ocre texte ») est réservée à cet usage. L'information n'est jamais portée par la couleur seule (statuts écrits en toutes lettres, étoiles accompagnées d'un texte « Note : 4 sur 5 »).

3. Typographie



Polices et hiérarchie typographique

- **Fraunces** (Google Fonts, licence SIL OFL) — titres, en graisses 500 et 700 : une serif chaleureuse qui rappelle les cartes de restaurant.
- **Inter** (licence SIL OFL) — textes, formulaires et interface, en graisses 400, 500 et 600 : très lisible à l'écran, chiffres tabulaires pour les prix et les horaires.

- Les deux polices sont **auto-hébergées** (fichiers WOFF2 dans `public/assets/fonts`) : aucune requête vers les serveurs de Google, ce qui évite le transfert de l'adresse IP des visiteurs hors UE (RGPD) et accélère l'affichage.
- Taille de base : 16 px, interligne 1,6. Les tailles de titre s'adaptent à l'écran (`clamp()`).

4. Composants

- **Boutons** : forme « pilule », bordeaux plein pour l'action principale, contour bordeaux pour l'action secondaire, rouge doux pour les actions destructrices (supprimer, annuler) avec confirmation.
- **Cartes de menu** : image 3:2, badge de thème, badge de régime (sauge), prix « à partir de » pour le minimum de personnes, bouton pleine largeur.
- **Encadré « Conditions du menu »** : fond ocre clair, bordure ocre de 2 px et icône d'alerte, placé à côté du bouton « Commander » et rappelé sur la page de commande, avec une case à cocher obligatoire — l'exigence « bien mettre en évidence les conditions » de l'énoncé.
- **Récapitulatif de prix** : panneau sombre (encre) qui contraste avec le formulaire et reste visible pendant la saisie.
- **Statuts de commande** : pastilles colorées *et* libellées (en attente, en cours, terminée, annulée, retour de matériel).
- **Focus clavier** : contour ocre de 3 px sur tous les éléments interactifs.

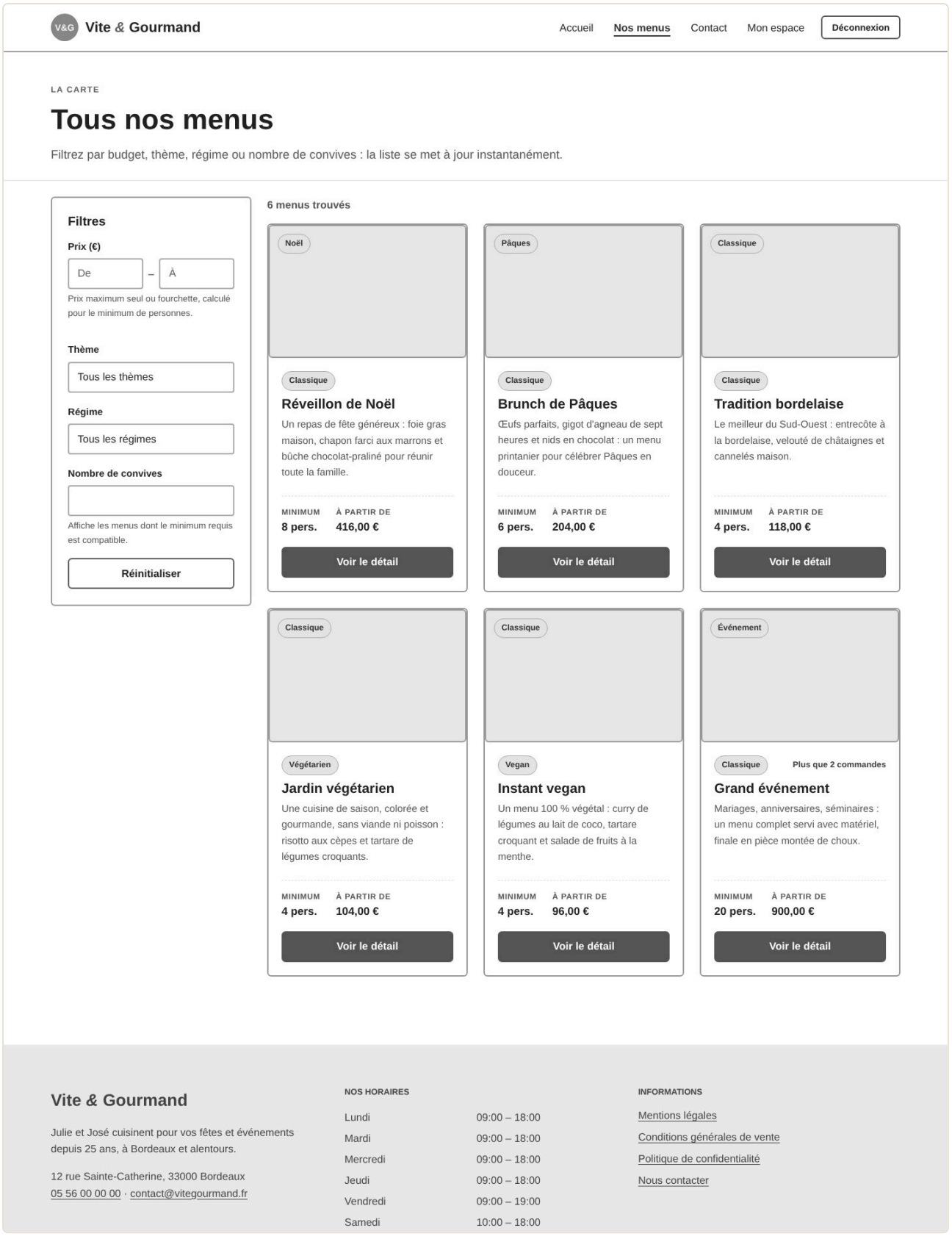
5. Maquettes

Les maquettes ont été conçues *mobile first* puis déclinées sur ordinateur. Les trois écrans retenus sont ceux du parcours principal : **accueil → vue globale des menus → commande**.

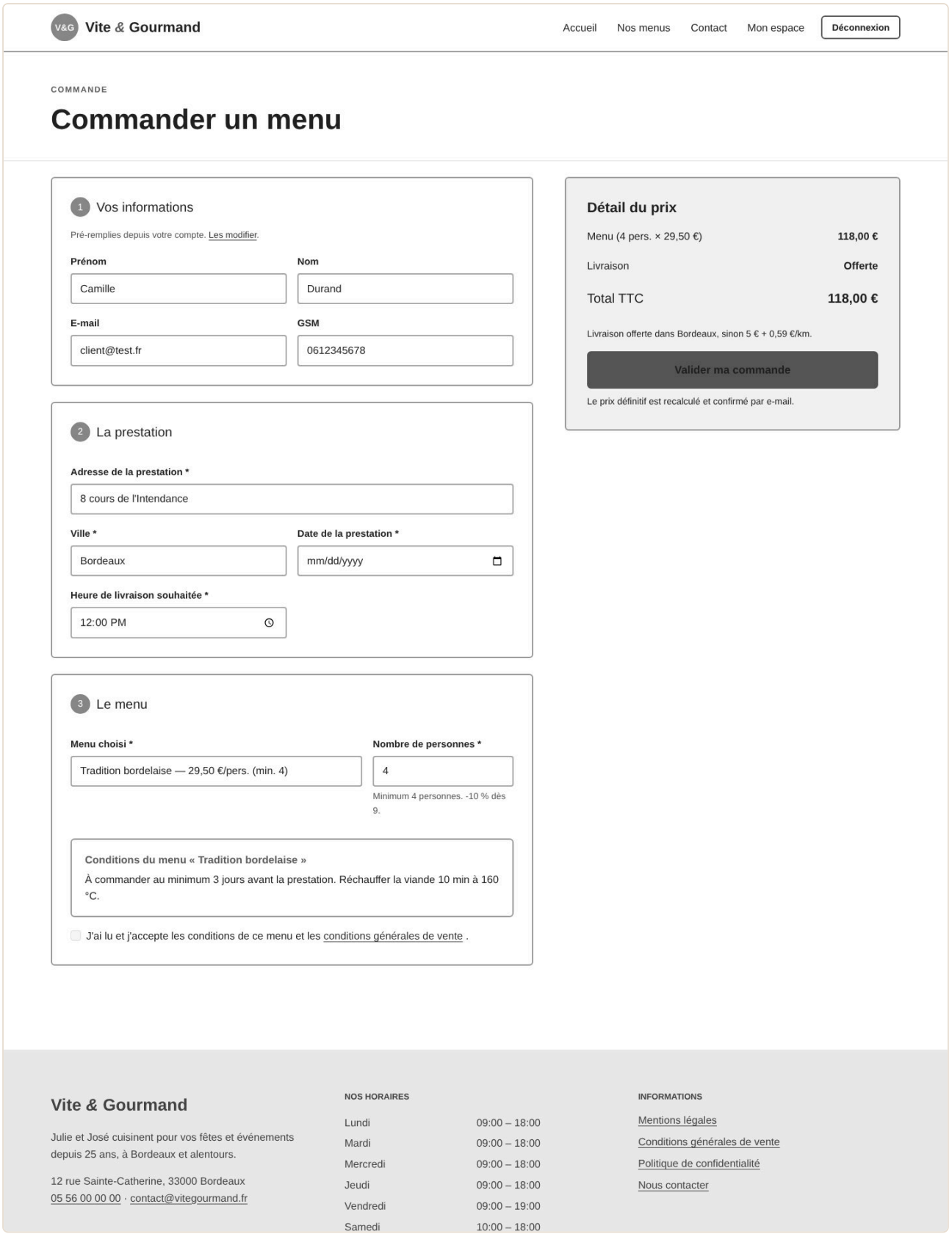
5.1 Wireframes — version bureau



Wireframe bureau — accueil



Wireframe bureau — vue globale des menus

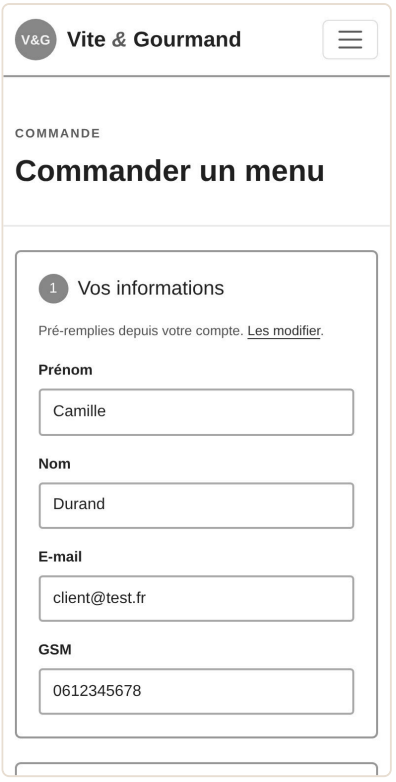


Wireframe bureau — commande

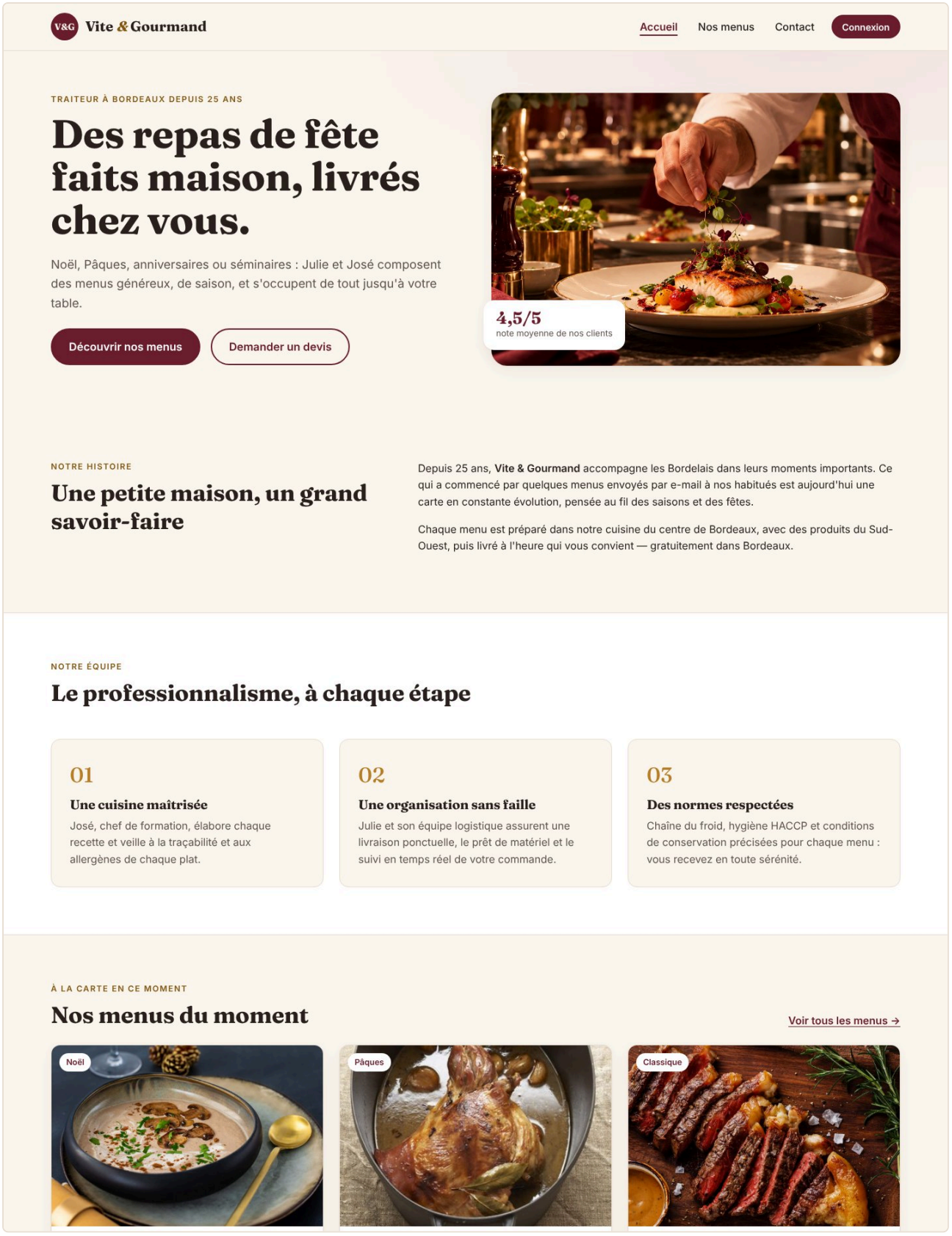
5.2 Wireframes — version mobile



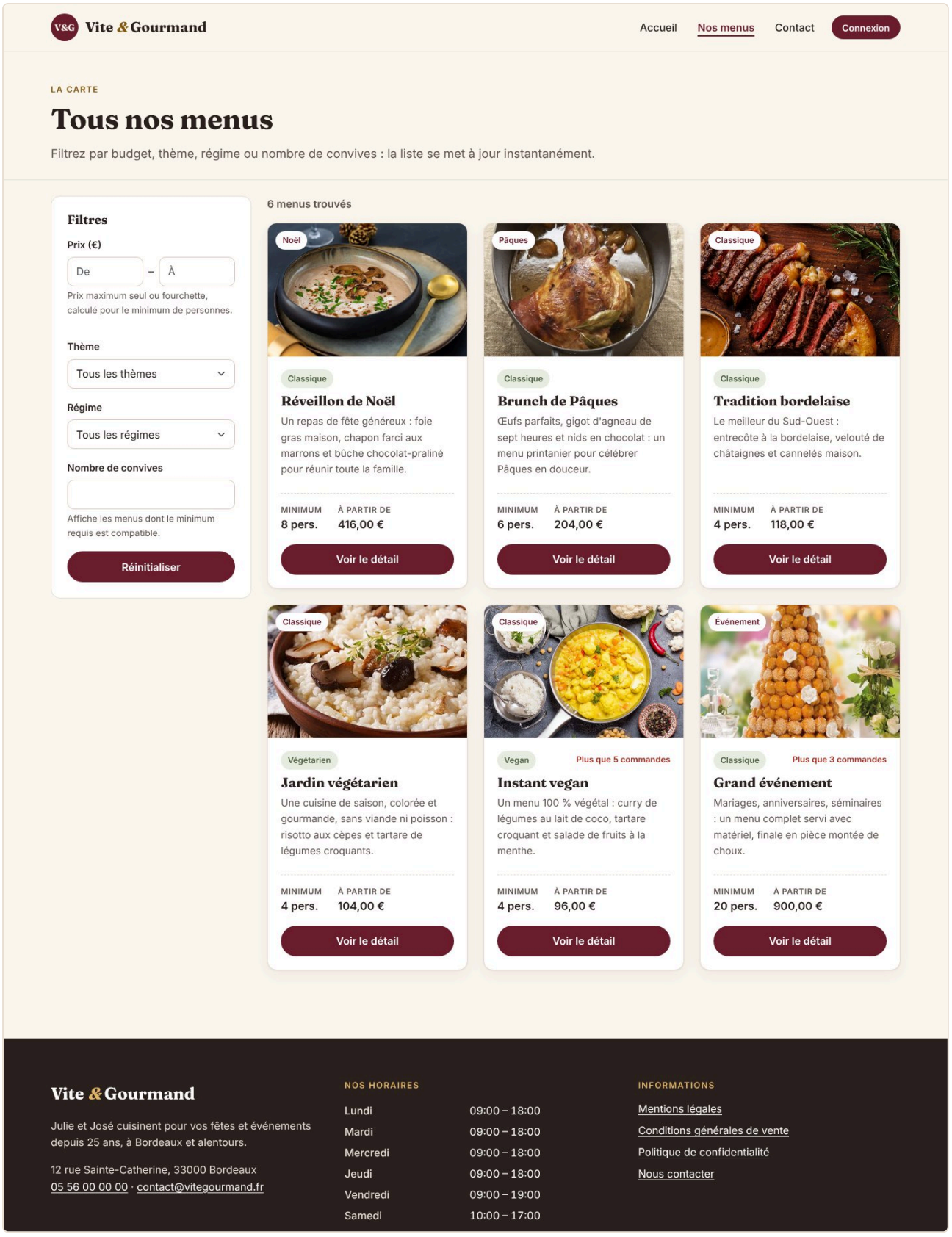




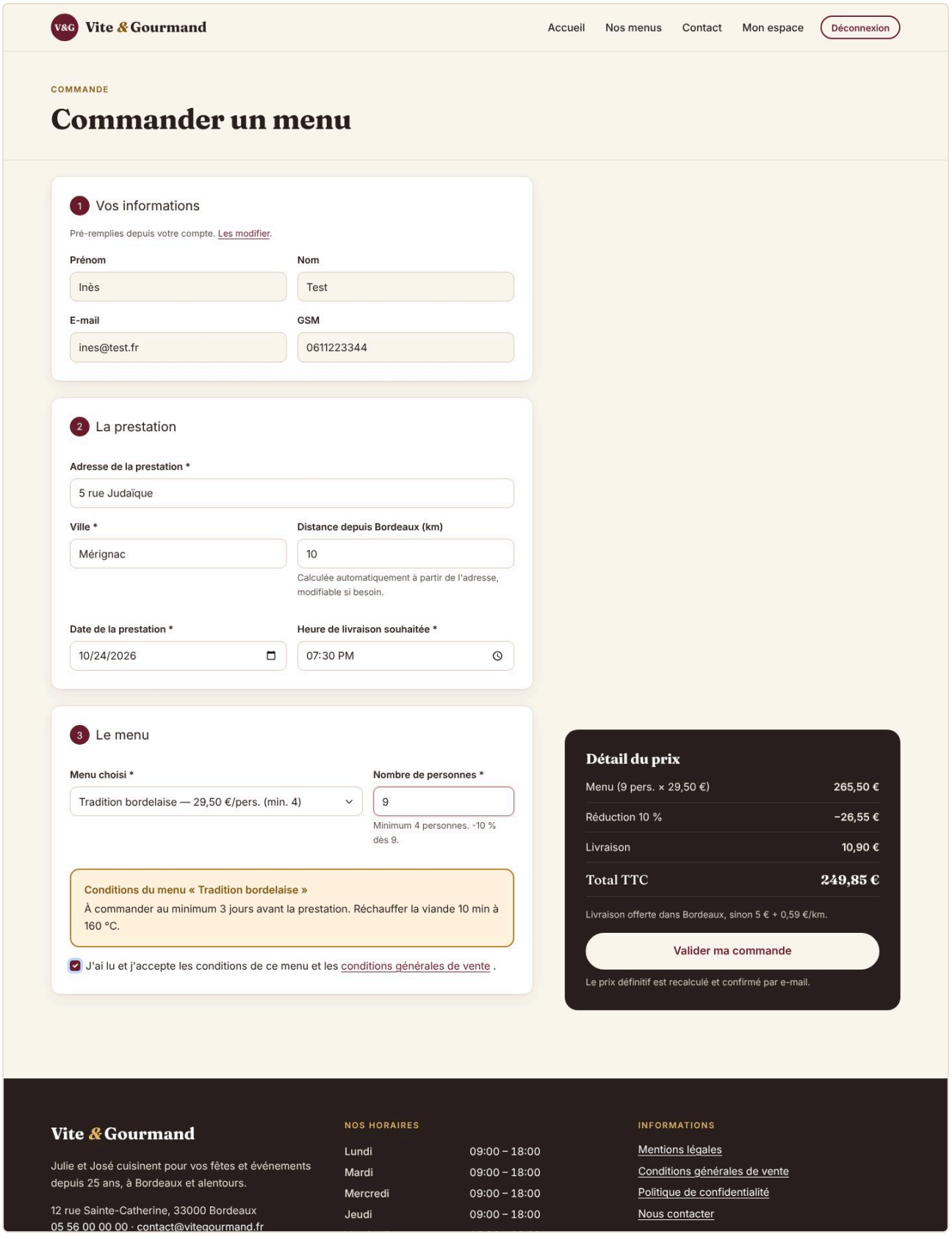
5.3 Mockups — version bureau



Mockup bureau — accueil

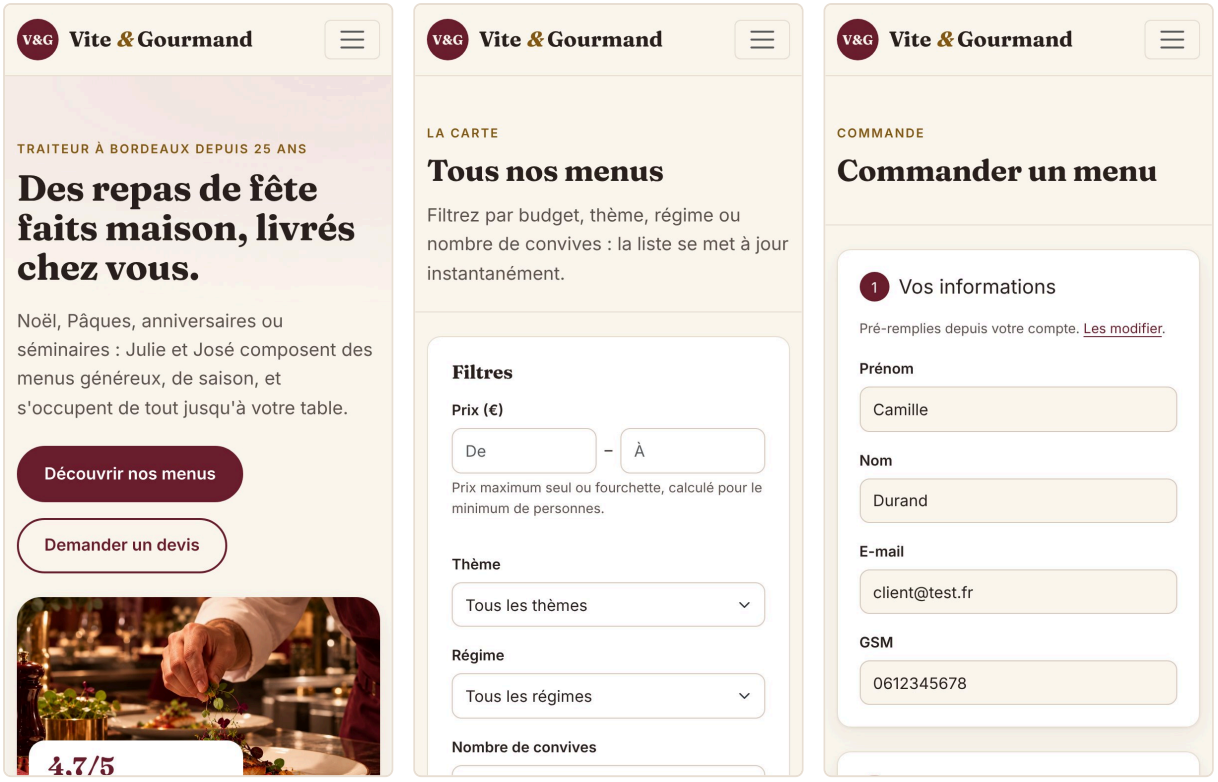


Mockup bureau — vue globale des menus



Mockup bureau — commande

5.4 Mockups — version mobile



V&G

Documentation technique

Choix techniques, modélisation, sécurité et déploiement

Projet : application web du traiteur Vite & Gourmand (Bordeaux)

Candidat : Mahmoud EL BRAIKI

Organisme : Studi — Évaluation en cours de formation

Date : Octobre 2026

Documentation technique

1. Réflexions initiales technologiques

1.1 Analyse du besoin

Julie et José n'ont aucune compétence informatique : l'application doit être **simple à administrer**, **peu coûteuse à héberger** et **maintenable** par une petite équipe. Le périmètre fonctionnel (catalogue filtrable, commande, suivi, back-office, statistiques) est celui d'une application de gestion classique, avec deux contraintes imposées : une **base relationnelle** et une **base non relationnelle**.

1.2 Choix retenus et justification

Couche	Choix	Pourquoi	Alternatives étudiées
Front-end	HTML 5, CSS 3, **Bootstrap 5.3**, JavaScript natif	Rendu serveur rapide et bien référencé ; Bootstrap fournit une grille responsive et des composants accessibles ; JavaScript natif moderne (ES2020+ : modules `import` / `export`, classes, `const` / `let`, fonctions fléchées, template literals, `async` / `await` avec `fetch`) suffit pour les filtres dynamiques et le calcul de prix, sans chaîne de build	React / Vue : apport faible pour quelques interactions, ajout d'un build et d'une API complète ; Tailwind : nécessite un build
Back-end	**PHP 8.1+ sans framework**, architecture **MVC légère** (contrôleur frontal, contrôleurs, repositories, services, vues)	PHP est disponible chez tous les hébergeurs mutualisés ; sans framework, chaque mécanisme de sécurité (CSRF, sessions, hachage, requêtes préparées) est écrit et maîtrisé, ce qui est l'objectif de l'évaluation	Symfony : très robuste mais lourd pour ce périmètre ; Node.js/Express : excellent mais hébergement moins courant chez les petits clients
Accès aux données SQL	**PDO** avec requêtes préparées	Protection native contre l'injection SQL, transactions, indépendant du SGBD	ORM Doctrine : surdimensionné ici
Base relationnelle	**MySQL 8 / MariaDB** (InnoDB)	Respect du MCD fourni, intégrité référentielle (clés étrangères, contraintes `CHECK`), transactions pour le stock ; disponible partout	PostgreSQL : équivalent, moins répandu en mutualisé

Couche	Choix	Pourquoi	Alternatives étudiées
Base NoSQL	**MongoDB** (Atlas en production)	Les statistiques sont des lectures agrégées : un document dénormalisé par commande (`menu_titre` , `montant` , `date` , `statut`) est idéal pour le pipeline `$match` / `$group` ; aucune jointure ; découplage des requêtes analytiques de la base transactionnelle	Redis : non adapté aux agrégations ; Firebase : verrouillage fournisseur
E-mails	**PHPMailer** (SMTP/TLS) + journal en développement	Envoi fiable et authentifié ; en local, les e-mails sont écrits dans `storage/logs/mail.log` pour être vérifiés sans serveur SMTP	`mail()` PHP : peu fiable, souvent bloqué
Graphiques	**Chart.js 4**	Léger, accessible (canvas avec libellé + tableau de données équivalent)	D3.js : plus complexe
Déploiement	**Docker + Fly.io**, MySQL managé, MongoDB Atlas	Image reproductible, HTTPS automatique, région Paris (`cdg`), offre gratuite / très faible coût	Heroku (payant), Vercel (pas de PHP natif), Azure (plus complexe)

Les bibliothèques front (Bootstrap, Chart.js) et les polices sont **auto-hébergées** : pas de dépendance à un CDN, une politique CSP stricte (`'self'` uniquement) et aucun transfert de données personnelles vers des tiers.

1.3 Repli NoSQL en local

Pour que l'application fonctionne sur un poste sans MongoDB (XAMPP), la classe `StatsStore` expose une interface unique et choisit le moteur au démarrage : MongoDB si `MONGO_URI` est défini et l'extension présente, sinon un **magasin de documents JSON** (`storage/nosql/commandes.json`) qui applique la même logique d'agrégation. En production, la source affichée sur la page Statistiques est « MongoDB ».

2. Configuration de l'environnement de travail

Outil	Version	Rôle
Visual Studio Code	1.9x	Éditeur (extensions : PHP Intelephense, PHP Debug, MySQL, MongoDB for VS Code, Prettier, axe Accessibility Linter)
PHP	8.3 (≥ 8.1)	Langage serveur, serveur de développement intégré `php -S`
MariaDB / MySQL	10.11 / 8.0	Base relationnelle (XAMPP possible)
MongoDB Community + mongosh	7.x	Base NoSQL locale (facultatif grâce au repli JSON)

Outil	Version	Rôle
Composer	2.x	Dépendances PHP (PHPMailer, mongodb/mongodb)
Git + GitHub	2.4x	Versionnement (branches main / develop / feature)
Navigateurs	Chrome, Firefox, Safari	Tests, outils de développement, mode responsive
Figma	—	Wireframes et maquettes
Trello / Notion	—	Gestion de projet (Kanban)
Docker + flyctl	—	Conteneurisation et déploiement
Lighthouse, WAVE, NVDA	—	Audits accessibilité et performance

Installation locale : voir le fichier `README.md` du dépôt (clonage, `composer install`, `.env`, import des scripts SQL, `php -S localhost:8000 -t public public/index.php`).

3. Architecture de l'application

```
Navigateur ──HTTP──▶ public/index.php (contrôleur frontal + routes)
                        │
                        ├─ src/Core        Env, Database (PDO), Auth, Csrf, View, Mailer, Upload
                        ├─ src/Controller  Home, Menu, Auth, Order, Account, Staff, Admin, Contact, Page
                        ├─ src/Repository  User, Menu, Plat, Commande, Avis, Ref   ──▶ MySQL
                        ├─ src/Service     PricingService, OrderStatus, StatsStore ──▶ MongoDB
                        └─ templates/      layout + pages + partiels (échappement systématique)
```

- **Un seul point d'entrée** (`public/index.php`) : seul le dossier `public/` est exposé par le serveur web.
- Les **contrôleurs** valident les entrées, vérifient les droits et appellent les **repositories** (SQL) et **services** (règles métier).
- Les **règles de prix** sont centralisées dans `PricingService` et recalculées côté serveur ; le JavaScript ne fait qu'afficher une estimation.
- Les **API JSON** `/api/menus` (filtres) et `/api/statistiques` (graphique) alimentent les mises à jour sans rechargement.
- Le **front-end** est organisé en **modules ES** (`public/assets/js/`) : `utils.js` regroupe les outils partagés (formatage monétaire, création d'éléments DOM sécurisée, sérialisation des filtres), importés par `menus.js` (classe `MenuFilter`), `order.js` (classe `OrderEstimator`), `stats.js` (classe `StatsDashboard`) et `app.js` (classe `PasswordChecker`, galerie, confirmations). Les appels réseau utilisent `async` / `await`, avec `AbortController` pour annuler une requête de filtrage devenue obsolète.

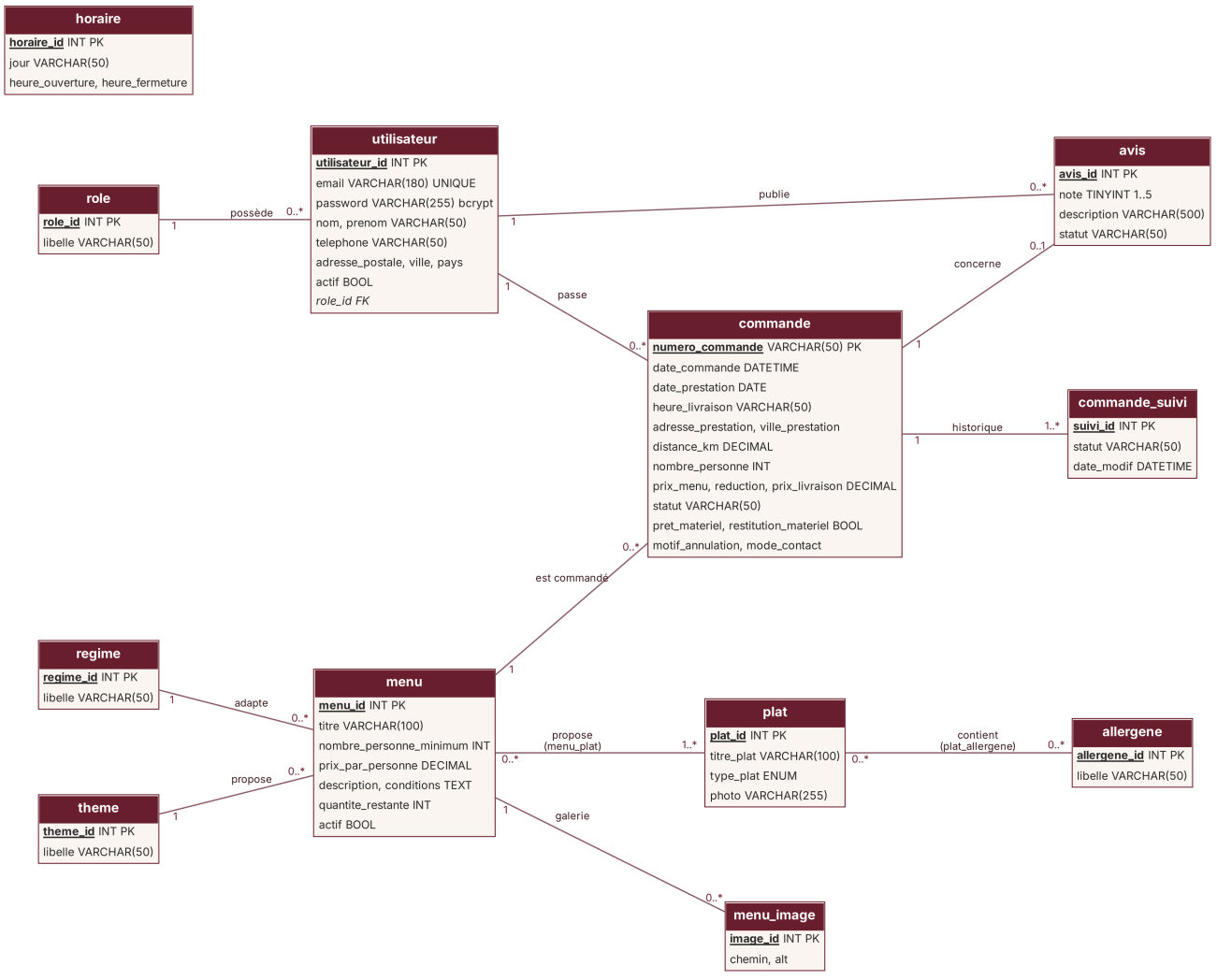
4. Modèle conceptuel de données

Le modèle reprend fidèlement le MCD de l'annexe 1 (utilisateur, rôle, commande, menu, thème, régime, plat, allergène, avis, horaire) et le complète par les tables nécessaires aux règles de gestion :

- `menu_image` — la **galerie d'images** d'un menu ;

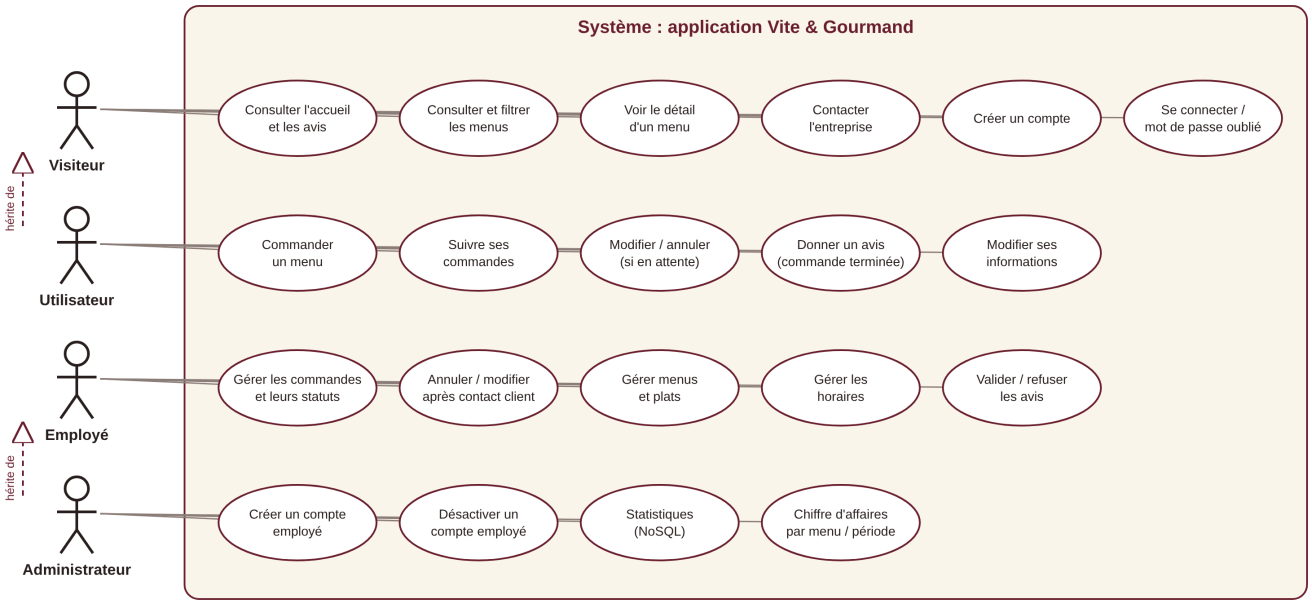
- `menu_plat` et `plat_allergene` — tables d'association des relations n,n *propose* et *contient* ;
- `commande_suivi` — l'**historique daté** des statuts affiché dans le suivi client ;
- `reinitialisation_mdp` — jetons de réinitialisation (stockés hachés en SHA-256, valables 1 h, usage unique) ;
- colonnes ajoutées : `utilisateur.nom` et `utilisateur.actif` (désactivation d'un employé), `commande.adresse/ville/distance/reduction/motif_annulation/mode_contact`, `menu.conditions/actif`.

Choix notables : `numero_commande` lisible (`VG-AAAAMMJJ-XXXX`) comme clé primaire ; montants en `DECIMAL` (pas de `DOUBLE`, pour éviter les erreurs d'arrondi monétaires) ; contraintes `CHECK` (note de 1 à 5, stock ≥ 0) ; photo des plats stockée comme **chemin de fichier** plutôt qu'en `BLOB`, pour alléger la base et laisser le serveur web et le cache navigateur servir les images.



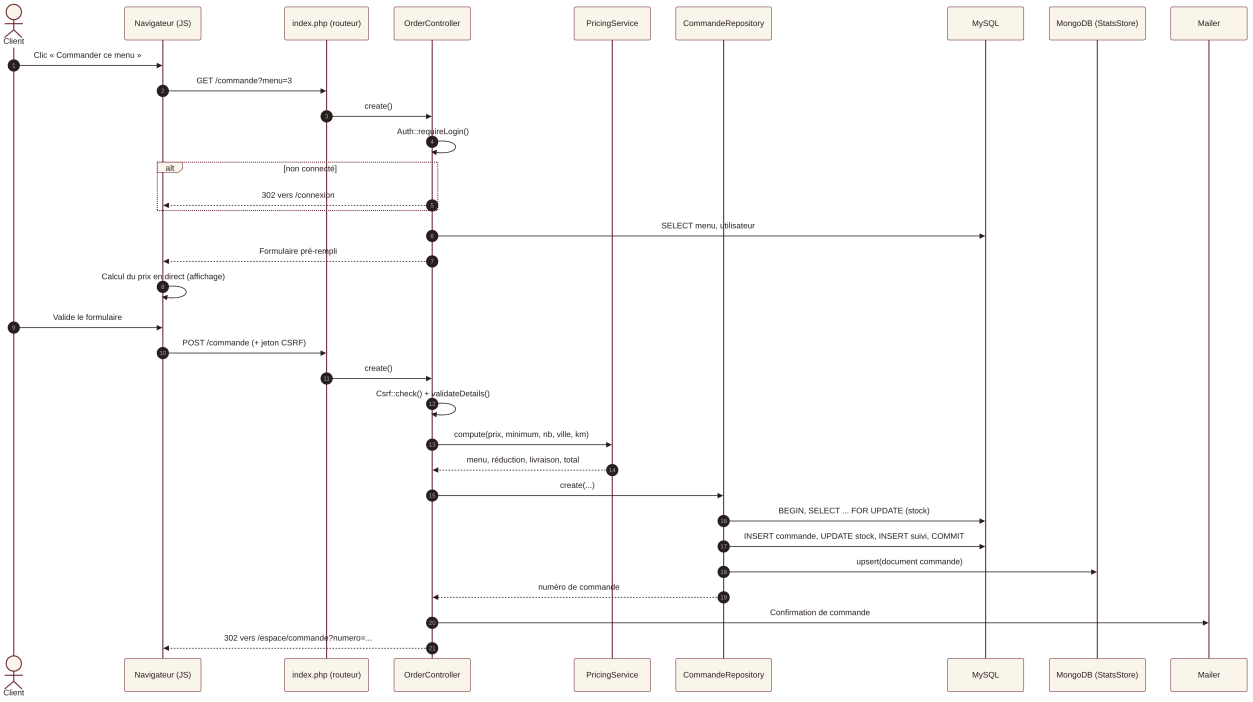
Modèle NoSQL — collection `commandes` (MongoDB) : un document par commande, alimenté à chaque création, modification ou changement de statut ; index sur `numero` (unique) et `{menu_id, date_commande}` ; schéma validé par `$jsonSchema` (`database/03_mongo_init.js`).

5. Diagramme de cas d’utilisation



L’utilisateur hérite des cas du visiteur ; l’administrateur hérite de tous les cas de l’employé.

6. Diagramme de séquence — commande d’un menu



Points clés : redirection vers la connexion si le visiteur n’est pas authentifié ; vérification du jeton CSRF ; recalcul serveur du prix ; transaction avec verrouillage `SELECT … FOR UPDATE` pour ne jamais vendre un menu dont le stock est épuisé ; écriture du document NoSQL ; e-mail de confirmation.

7. Sécurité

Menace	Mesure mise en place
Injection SQL	PDO, requêtes **préparées** exclusivement, `ATTR_EMULATE_PREPARES = false` ; filtres convertis en types (int / float)
XSS	Fonction `e()` (`htmlspecialchars` , `ENT_QUOTES` , UTF-8) sur toutes les sorties ; côté JS, insertion via `textContent` uniquement ; **Content-Security-Policy** stricte (`default-src 'self'` , aucun script en ligne)
CSRF	Jeton aléatoire de 256 bits par session, vérifié par `hash_equals` sur **tous** les formulaires POST (y compris la déconnexion) ; cookie `SameSite=Lax`
Vol de session	Cookie `HttpOnly` , `Secure` en HTTPS, `use_strict_mode` , `session_regenerate_id()` à la connexion
Mots de passe	Hachage **bcrypt** (coût 12) avec `password_hash` ; politique imposée côté serveur (10 caractères, majuscule, minuscule, chiffre, caractère spécial) ; jamais transmis par e-mail
Force brute	Verrouillage de 5 minutes après 5 échecs ; temps de réponse homogène même si l'e-mail n'existe pas
Énumération de comptes	Message identique pour « mot de passe oublié », que le compte existe ou non
Contrôle d'accès	`Auth::requireRole()` sur chaque action ; vérification de propriété des commandes (anti-IDOR) ; compte désactivé déconnecté immédiatement ; **création d'administrateur impossible** (rôle forcé côté serveur)
Téléversement	Type MIME réel vérifié (`finfo`), 2 Mo max, nom aléatoire, exécution de scripts interdite dans `uploads/`
Redirection ouverte	Après connexion, seules les URL internes sont acceptées
Spam	Champ « pot de miel » sur le formulaire de contact
Base de données	Utilisateur MySQL applicatif avec privilèges limités (`SELECT, INSERT, UPDATE, DELETE`) ; identifiants dans `.env` , exclu de Git
En-têtes HTTP	`X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy` , `Permissions-Policy` , HSTS en HTTPS ; `expose_php = Off`
Erreurs	En production, messages génériques et détail uniquement dans les journaux

RGPD : minimisation des données collectées, consentement explicite à l'inscription, politique de confidentialité, droit de rectification depuis l'espace client, avis publiés avec prénom et initiale seulement, statistiques NoSQL sans donnée personnelle, aucun cookie tiers (seul le cookie de session strictement nécessaire est utilisé).

8. Accessibilité (RGAA 4.1)

- Langue déclarée (`lang="fr"`), titres de page uniques, hiérarchie de titres cohérente, structure `header / nav / main / footer`.
- **Lien d'évitement** « Aller au contenu principal », navigation complète au clavier, **focus visible** (contour de 3 px).
- Tous les champs ont un `label` associé ; groupes de champs dans des `fieldset / legend` ; champs obligatoires signalés en texte.
- Contrastes AA vérifiés (voir charte) ; l'information n'est jamais transmise par la couleur seule.
- Images : alternatives textuelles pertinentes ; images décoratives avec `alt=""`.
- Contenus dynamiques annoncés aux lecteurs d'écran (`aria-live` sur le nombre de résultats des filtres, sur le récapitulatif de prix et les messages) ; `aria-current` sur la page active ; `aria-pressed` sur les vignettes de la galerie.
- Tableaux avec `caption` et en-têtes `scope` ; graphique doublé d'un tableau de données.
- Respect de `prefers-reduced-motion` ; mise en page sans défilement horizontal jusqu'à 320 px.

9. Tests réalisés

Une recette automatisée de bout en bout (Playwright) couvre **44 vérifications** : filtres dynamiques, redirection du visiteur vers la connexion, politique de mot de passe, calcul du prix (réduction et livraison), modification et annulation par le client, refus d'accès entre rôles (403), annulation impossible sans contact client, workflow complet des statuts et du retour de matériel, e-mails envoyés, avis et modération, création et désactivation d'un employé, statistiques NoSQL et filtres, absence de défilement horizontal sur mobile. Toutes les vérifications sont au vert.

10. Documentation du déploiement

10.1 Démarche

L'application est **en ligne sur alwaysdata** (hébergement mutualisé, PHP 8.3, MariaDB) à l'adresse **https://elbraiki.alwaysdata.net** — procédure suivie détaillée en 10.4. Le dépôt fournit aussi un déploiement conteneurisé (image Docker PHP 8.3 + Apache sur Fly.io, MySQL managée, MongoDB Atlas) décrit ci-dessous.

10.2 Étapes

1. **Base relationnelle** — créer une base MySQL 8 managée (ex. Aiven, Railway ou PlanetScale-compatible), puis importer `database/01_schema.sql` (sans les deux premières lignes `DROP/CREATE DATABASE` si l'hébergeur impose le nom de base) et `database/02_data.sql`. Créer un utilisateur applicatif aux droits limités.
2. **Base NoSQL** — créer un cluster gratuit **MongoDB Atlas** (région Europe), un utilisateur, autoriser l'accès réseau, puis exécuter `mongosh "<URI>" database/03_mongo_init.js`.
3. **SMTP** — créer un compte d'envoi transactionnel (Brevo, Mailjet…) et récupérer les identifiants SMTP.
4. **Fly.io** — installer `flyctl`, se connecter (`fly auth login`), puis à la racine du projet :
   - `fly launch --no-deploy` (reprend `fly.toml`, région `cdg` Paris) ;

- `fly secrets set DB_HOST=… DB_USER=… DB_PASS=… MONGO_URI=… MAIL_HOST=… MAIL_USER=… MAIL_PASS=… APP_URL=https://vite-gourmand.fly.dev` ;
- `fly deploy` : construction de l'image (`Dockerfile` : extensions `pdo_mysql` et `mongodb`, `composer install --no-dev`, racine web sur `public/`, `mod_rewrite`) puis mise en ligne.

5. **Vérifications** — HTTPS forcé (`force_https`), page d'accueil, connexion avec chaque rôle, commande test, réception des e-mails, page Statistiques indiquant la source « MongoDB », en-têtes de sécurité (`curl -I`).
6. **Mises à jour** — merge de `develop` dans `main` après tests, puis `fly deploy` ; en cas de problème, `fly releases` et retour à la version précédente.

10.3 Points d'attention

- Les variables sensibles ne sont **jamais** dans le dépôt : `.env` est ignoré par Git, les secrets sont gérés par `fly secrets`.
- Le disque des machines Fly est éphémère : les images téléversées depuis le back-office doivent, en évolution, être stockées sur un stockage objet (S3 / Tigris). Les images du jeu de données sont incluses dans l'image.
- Alternative mutualisée (ex. alwaysdata, o2switch) : déposer les fichiers par Git/SFTP, pointer la racine du site sur `public/`, créer `.env`, importer les scripts SQL — l'application fonctionne sans Docker.

10.4 Mise en ligne effective (alwaysdata)

1. Compte alwaysdata (offre gratuite) ; base MariaDB `elbraiki_ecf` créée depuis l'interface d'administration.
2. Site de type PHP 8.3, répertoire racine `~/www/vite-gourmand/public/` : seul `public/` est exposé.
3. En SSH : `git clone` du dépôt GitHub dans `~/www`, puis création du fichier `.env` sur le serveur (`APP_ENV=prod`, identifiants MySQL) — jamais versionné.
4. Import de `01_schema.sql`, `02_data.sql` et `04_photos.sql` (lignes `CREATE DATABASE` / `USE` retirées : le nom de base est imposé par l'hébergeur).
5. `php bin/sync-nosql.php` : initialisation de la base NoSQL en repli JSON (MongoDB n'est pas proposé par l'hébergeur).
6. Vérifications : HTTPS, accueil, filtres, commande, espaces utilisateur, employé et administrateur.
7. Mises à jour : merge de `develop` dans `main`, puis `git pull` dans `~/www/vite-gourmand`.

V&G

Gestion de projet

Méthode, planning, backlog et workflow Git

Projet : application web du traiteur Vite & Gourmand (Bordeaux)

Candidat : Mahmoud EL BRAIKI

Organisme : Studi — Évaluation en cours de formation

Date : Octobre 2026

Gestion de projet

1. Méthode

Le projet a été mené selon une approche **agile de type Kanban**, adaptée à un développement en solo avec un client (Julie et José) et un chef de projet FastDev :

- un **tableau Kanban** (Trello / Notion) à cinq colonnes : *Backlog*, *À faire*, *En cours*, *En test*, *Terminé* ;
- chaque besoin de l'énoncé est découpé en **user stories** (« En tant que… je veux… afin de… ») avec des critères d'acceptation, une étiquette de priorité (MoSCoW) et une étiquette d'espace (visiteur, utilisateur, employé, admin, transverse) ;
- des **itérations d'une semaine** ; en fin d'itération, démonstration au chef de projet et réajustement du backlog ;
- règle « une carte = une branche Git » : chaque carte passée *En cours* correspond à une branche `feature/…` créée depuis `develop` ; elle passe *En test* à l'ouverture de la fusion et *Terminé* une fois fusionnée et testée.

2. Planning (≈ 70 h)

Itération	Contenu	Charge
0 — Cadrage	Lecture de l'énoncé, analyse des besoins, user stories, choix techniques, environnement	6 h
1 — Conception	MCD complété, diagrammes (cas d'utilisation, séquence), charte graphique, wireframes et maquettes	10 h
2 — Socle	Base de données, architecture MVC, sécurité (sessions, CSRF, PDO), layout responsive	10 h
3 — Visiteur	Accueil, vue globale et filtres dynamiques, détail d'un menu, contact, pages légales	9 h
4 — Utilisateur	Inscription, connexion, mot de passe oublié, commande et calcul du prix, espace client, avis	12 h
5 — Back-office	Espace employé (commandes, statuts, menus, plats, horaires, avis), espace administrateur, statistiques NoSQL	12 h

Itération	Contenu	Charge
6 — Qualité et livraison	Recette, accessibilité (RGAA), corrections, déploiement, documentation	11 h

3. Backlog (extrait)

ID	User story	Priorité	Espace
US01	En tant que visiteur, je veux voir la présentation de l'entreprise et les avis validés afin d'avoir confiance	Must	Visiteur
US02	En tant que visiteur, je veux filtrer les menus (prix, fourchette, thème, régime, personnes) sans rechargement	Must	Visiteur
US03	En tant que visiteur, je veux voir le détail d'un menu, ses plats, allergènes et conditions bien visibles	Must	Visiteur
US04	En tant que visiteur, je veux créer un compte avec un mot de passe sécurisé et recevoir un e-mail de bienvenue	Must	Visiteur
US05	En tant qu'utilisateur, je veux me connecter et réinitialiser mon mot de passe par e-mail	Must	Utilisateur
US06	En tant qu'utilisateur, je veux commander un menu avec mes informations pré-remplies et voir le détail du prix	Must	Utilisateur
US07	En tant qu'utilisateur, je veux modifier ou annuler ma commande tant qu'elle n'est pas acceptée	Must	Utilisateur

ID	User story	Priorité	Espace
US08	En tant qu'utilisateur, je veux suivre les états de ma commande avec date et heure	Must	Utilisateur
US09	En tant qu'utilisateur, je veux noter (1 à 5) et commenter une commande terminée	Must	Utilisateur
US10	En tant qu'employé, je veux filtrer les commandes par statut ou client et mettre à jour leur statut	Must	Employé
US11	En tant qu'employé, je dois indiquer le mode de contact et le motif avant de modifier ou annuler une commande	Must	Employé
US12	En tant qu'employé, je veux gérer les menus, plats, allergènes et horaires	Must	Employé
US13	En tant qu'employé, je veux valider ou refuser les avis	Must	Employé
US14	En tant qu'administrateur, je veux créer et désactiver des comptes employés	Must	Admin
US15	En tant qu'administrateur, je veux comparer les commandes par menu (graphique, NoSQL) et le chiffre d'affaires par période	Must	Admin
US16	En tant que client, je veux une application accessible (RGAA) et conforme au RGPD	Must	Transverse
US17	En tant que client, je veux l'application déployée et en ligne à la livraison	Must	Transverse

ID	User story	Priorité	Espace
US18	En tant qu'utilisateur, je veux que la distance de livraison soit estimée automatiquement depuis mon adresse	Should	Utilisateur

Le backlog complet est fourni au format CSV (`docs/gestion-projet-notion.csv`), importable directement dans Notion ou dans un tableau Trello.

4. Gestion des versions (Git)

- `main` : version stable, déployée en production ; étiquette `v1.0.0` à la livraison.
- `develop` : branche d'intégration.
- `feature/<nom>` : une branche par fonctionnalité (`feature/base-de-donnees`, `feature/menus`, `feature/authentification`, `feature/commande`, `feature/espace-utilisateur`, `feature/espace-employe`, `feature/espace-admin`, `feature/contact-legal`, `feature/deploiement`…), fusionnée dans `develop` après test (`--no-ff` pour conserver l'historique), puis `develop` fusionnée dans `main`.
- Messages de commit conventionnels (`feat:`, `fix:`, `docs:`, `chore:`).

5. Suivi et qualité

- **Définition de « terminé »** : critères d'acceptation validés, tests de recette passés, aucune erreur console, contrôle clavier et contraste, code relu et fusionné.
- **Recette automatisée** (Playwright) rejouée avant chaque fusion dans `main`.
- **Risques identifiés** : délai (priorisation MoSCoW), hébergement (déploiement anticipé dès l'itération 2), absence de MongoDB sur certains postes (repli JSON), envoi d'e-mails (journal en développement, SMTP en production).